Foundations of Artificial Intelligence

Part III: Knowledge Representation & Inference

Department of Computer Science

Lecture Roadmap

1. Why Knowledge Representation?
2. Logic Foundations
3. Inference in Knowledge Systems
4. Rules, Semantic Networks, Ontologies
5. Putting It Together
6. Assessment and Wrap-Up

Learning Objectives

By the end of this lecture, you should be able to:

- Explain why representation choices determine what an AI system can infer.
- Model facts and rules in propositional and first-order logic.
- Apply inference ideas: entailment, chaining, and resolution at a conceptual level.
- Relate rule systems, semantic networks, and ontologies to real AI applications.
- Analyze the strengths/limits of each representation for practical tasks.

From Data to Knowledge

AI systems need more than raw data:

- **Data:** isolated observations (e.g., temperature is $39^{\circ}\mathrm{C}$).
- **Information:** structured observations (patient has fever + cough).
- **Knowledge:** reusable, inferable structure (if fever and cough, suspect infection).

Book perspective: a representation is useful only if it supports effective inference.

Real-World Motivation

- **Clinical decision support:** map symptoms to diagnostic hypotheses.
- **Fraud detection:** infer suspicious patterns from transaction relations.
- **Customer support bots:** apply business rules consistently.
- **Industrial maintenance:** reason over component dependencies and failure rules.

Common requirement: **explicit knowledge** + **traceable reasoning**.

What Makes a Good Representation?

We judge KR by four practical criteria:

1. **Expressiveness:** Can it describe what matters?
2. **Inference support:** Can we derive useful conclusions?
3. **Computational cost:** Can we reason in acceptable time?
4. **Maintainability:** Can humans update and validate it?

Propositional Logic: Core Idea

Propositional logic is the **simplest formal language** for expressing and reasoning about truth.

- **Atomic propositions** are the basic building blocks — indivisible statements that are either *true* or *false* (e.g., P = "It is raining", Q = "The road is wet").
- **Connectives** combine propositions into complex formulas: $\neg$ (not), $\wedge$ (and), $\vee$ (or), $\Rightarrow$ (if-then), $\Leftrightarrow$ (if and only if).
- A **model** (or interpretation) assigns True/False to every atom; a formula is evaluated under this assignment.
- **Limitation:** Propositions are *flat* — no objects, no variables, no structure inside. Every distinct fact needs its own symbol.

Smart-home example:

$$(\text{Smoke} \wedge \text{Heat}) \Rightarrow \text{Alarm}$$

If both sensors trigger, sound the alarm. This single rule replaces ad-hoc if/else code and is *inspectable, explainable, and reusable*.

Semantics (1/2): Core Concepts

Semantics defines *what a formula means* — its truth value across all possible worlds (models).

- A **model** (or interpretation) assigns *True* or *False* to every atomic proposition. Think of it as one possible "state of the world."
- If *KB* uses n atoms, there are 2^n possible models to consider. With 3 atoms, that is 8 worlds; with 10 atoms, 1024 worlds.
- A formula is **true under a model** if the assignment makes it evaluate to True.
- A model **satisfies** *KB* if *every* sentence in *KB* is true under it.

Semantics is what separates logic from mere symbol manipulation — it grounds every symbol in meaning.

Semantics (2/2): Truth Table Example

Example: Let $P =$ "it is raining" and $Q =$ "the road is wet."

We evaluate the formula $P \Rightarrow Q$ across all four possible models:

P	Q	$P \Rightarrow Q$	Interpretation
T	T	T	Rained and road is wet — rule holds.
T	F	F	Rained but road is dry — rule violated!
F	T	T	No rain, road wet (other cause) — rule holds.
F	F	T	No rain, road dry — rule holds vacuously.

The implication fails in **only one** model — when the antecedent is true but the consequent is false.

A formula is semantically meaningful precisely because we can identify which worlds satisfy it.

Entailment

Entailment ($KB \models \alpha$) is the central concept of logical reasoning:

$$KB \models \alpha \quad \iff \quad \text{every model satisfying } KB \text{ also satisfies } \alpha$$

Intuitively: *"If everything in KB is true, then α must be true too — no escape."*

Example:

- $KB = \{P \Rightarrow Q,\ P\}$
- In every model where both P and $P \Rightarrow Q$ hold, Q must hold.
- Therefore: $KB \models Q$.

Entailment vs. Derivation — a critical distinction:

- $KB \models \alpha$ (entailment) — a *semantic* claim: about truth across all worlds.
- $KB \vdash \alpha$ (derivation) — a *syntactic* claim: produced by applying inference rules.
- **Sound** system: $KB \vdash \alpha \ \Rightarrow \ KB \models \alpha$ (never derives falsehoods)
- **Complete** system: $KB \models \alpha \ \Rightarrow \ KB \vdash \alpha$ (misses nothing true)

Entailment is the gold standard. Inference procedures try to match it efficiently.

Validity

A sentence is **valid** (a *tautology*) if it is true in **every possible model** — regardless of what the atoms mean.

Examples of valid sentences:

- $P \vee \neg P$ — always true (law of excluded middle).
- $(P \Rightarrow Q) \Leftrightarrow (\neg Q \Rightarrow \neg P)$ — contrapositive equivalence.
- $\neg(P \wedge \neg P)$ — contradiction can never hold (law of non-contradiction).

Three-way classification of sentences:

Type	True in	Example
Valid (tautology)	Every model	$P \vee \neg P$
Satisfiable	At least one model	$P \wedge Q$
Unsatisfiable	No model	$P \wedge \neg P$

Why validity matters for inference:
Resolution provers prove $KB \models \alpha$ by showing $KB \wedge \neg\alpha$ is *unsatisfiable* — turning an entailment question into a satisfiability check.

Mini Propositional Example

Scenario: A delivery system has two rules about road conditions.

Knowledge Base (KB):

(1) $R \Rightarrow W$ ("If it rained, the road is wet.")

(2) $W \Rightarrow D$ ("If the road is wet, deliveries are delayed.")

(3) R ("It rained." — observed fact)

Query: Is D (delivery delayed) entailed?

Step-by-step inference via Modus Ponens:

$$\frac{R \Rightarrow W, \quad R}{W} \quad \text{then} \quad \frac{W \Rightarrow D, \quad W}{D}$$

Result: $KB \vdash D$, and since the system is sound, $KB \models D$.

*Insight: Modus Ponens fires rules like a chain. Each step produces a new fact that can trigger the next rule. This is **forward chaining** in action.*

CNF and SAT (1/2): Core Idea

CNF (Conjunctive Normal Form) is a standard way to write formulas.

- **Literal:** A or $\neg A$
- **Clause:** OR of literals, e.g., $(\neg A \vee B)$
- **CNF:** AND of clauses

Example CNF:

$$(\neg A \vee B) \wedge (\neg B \vee C) \wedge A$$

Why CNF?

- SAT solvers are optimized for this shape.
- Uniform format makes reasoning faster in practice.

CNF and SAT (2/2): SAT in Practice

SAT question: Is there *any* assignment that makes the CNF true?

$$(\neg A \vee B) \wedge (\neg B \vee C) \wedge A$$

Try $A{=}T, B{=}T, C{=}T$:

- $(\neg A \vee B) = (F \vee T) = T$
- $(\neg B \vee C) = (F \vee T) = T$
- $A = T$

So this formula is **satisfiable**.

Where SAT is used:

- planning,
- hardware/software verification,
- scheduling and timetabling.

First-Order Logic (FOL) (1/2): Why We Need It

Problem with propositional logic:

- It cannot naturally talk about *objects* and *relations*.
- You need one symbol per fact (not scalable).

Example idea: "Every patient with fever needs rest."

- In propositional logic: many separate symbols/rules.
- In FOL: one reusable rule for all patients.

Goal of FOL: write compact rules that generalize to many entities.

First-Order Logic (FOL) (2/2): Building Blocks

FOL adds structure using these elements:

- **Constants:** Ali, London, DrSmith
- **Variables:** x, y, z
- **Predicates:** $\mathit{Doctor}(x)$, $\mathit{Treats}(x, y)$
- **Quantifiers:** $\forall$ (for all), $\exists$ (there exists)

Example rule:

$$\forall p \, (\mathit{Patient}(p) \land \mathit{HasFever}(p) \Rightarrow \mathit{NeedsRest}(p))$$

One sentence applies to every patient.

FOL Example (1/2): Book-Style

Classic example:

$$\forall x \, (Human(x) \Rightarrow Mortal(x))$$

$$Human(Socrates)$$

Therefore:

$$Mortal(Socrates)$$

This is: apply a general rule to one specific object.

FOL Example (2/2): Real-World

Healthcare rule:

$$\forall p \, (\mathit{HasFlu}(p) \Rightarrow \mathit{NeedsRest}(p))$$

$$\mathit{HasFlu}(\mathit{Ali})$$

Therefore:

$$\mathit{NeedsRest}(\mathit{Ali})$$

Takeaway: same pattern as Socrates, but now in a practical domain.

Unification (1/2): Intuition

Unification means: find substitutions that make two expressions match.

Example:

$$\mathit{Treats}(\mathit{DrSmith}, x) \quad \text{and} \quad \mathit{Treats}(\mathit{DrSmith}, \mathit{Sam})$$

Unifier:

$$\theta = \{x/\mathit{Sam}\}$$

After substitution, both expressions are identical.

Unification (2/2): Success and Failure

Success:

$$Knows(x, y),\ Knows(Alice, Bob) \Rightarrow \theta = \{x/Alice,\ y/Bob\}$$

Failure:

$$Treats(DrSmith, Sam),\ Treats(DrJones, Sam)$$

cannot unify because $DrSmith \neq DrJones$.

Why important:

- backward chaining,
- resolution,
- Prolog-style rule matching.

Inference: Big Picture

Inference answers: *What new facts follow from what we already know?*

Desirable properties:

- **Soundness:** never derive false conclusions.
- **Completeness:** derive every true entailed conclusion (within logic limits).
- **Efficiency:** finish in practical time.

Forward vs Backward Chaining

Forward Chaining

- Data-driven
- Fire rules from known facts
- Good for streaming/event systems

Backward Chaining

- Goal-driven
- Start from query, prove prerequisites
- Good for diagnosis/query answering

Resolution (Conceptual)

Resolution works by contradiction:

1. Convert KB and $\neg Query$ into CNF clauses.
2. Repeatedly resolve complementary literals.
3. Derive empty clause to prove entailment.

Why useful:

- Uniform rule for automated theorem proving.
- Foundation for many symbolic reasoners.

Production Rules

Rules encode procedural domain knowledge:

IF condition THEN conclusion/action

Example (e-commerce):

- IF customer is premium AND cart value > 500 THEN shipping $=$ free.
- IF payment fails 3 times THEN flag transaction.

Rule-Based System Architecture

- **Knowledge Base:** facts + rules.
- **Working Memory:** current case-specific facts.
- **Inference Engine:** chooses and applies rules.
- **Explanation Layer:** why a decision was reached.

Strength: interpretable decisions. Limitation: rule maintenance can grow hard at scale.

Semantic Networks (Graph View of Knowledge)

- Nodes represent concepts/instances.
- Edges represent relations: `is-a`, `part-of`, `causes`, etc.
- Supports inheritance-like reasoning.

Example:

- `Bird is-a Animal`
- `Canary is-a Bird`
- infer: `Canary is-a Animal`

Logic vs Rules vs Semantic Nets

Approach	Main Strength	Typical Use	Main Limitation
Logic (PL/FOL)	Precise semantics	Formal reasoning, verification	Can become computationally heavy
Production Rules	Simple expert knowledge encoding	Policy, diagnosis, decision support	Rule explosion/conflicts
Semantic Networks	Intuitive relationship structure	Taxonomies, concept maps, explainability	Semantics can be less strict

Ontologies: Basic Idea

Ontology = explicit conceptual model of a domain:

- **Classes:** Patient, Disease, Medication
- **Relations:** hasSymptom, treatedBy, contraindicatedWith
- **Instances:** specific entities (e.g., Patient_102)

Purpose:

- Shared vocabulary across teams/systems.
- Better interoperability and data integration.

End-to-End KR Pipeline (Practical)

1. Define domain concepts and relations.
2. Choose representation (logic/rules/net/ontology blend).
3. Encode facts and rules.
4. Select inference strategy.
5. Validate outputs with domain experts.
6. Iterate as domain evolves.

Common Pitfalls in KR Projects

- **Over-modeling:** too much detail too early.
- **Ambiguous predicates:** unclear symbol meaning.
- **Hidden assumptions:** not encoded explicitly.
- **No governance:** rules drift and become inconsistent.

Mitigation: naming conventions, review cycles, test queries, and versioning.

Assessment Q1 (Conceptual)

Question: Why is "representation" as important as the inference algorithm?

Expected discussion points:

- It determines what can be expressed.
- It constrains what can be inferred efficiently.
- Poor representation can make good inference unusable.

Assessment Q2 (Logic Modeling)

Convert to propositional statements and infer:

- If server overload then alert admin.
- If alert admin then open incident ticket.
- Server overload occurred.

Ask students: What follows logically? What does not necessarily follow?

Assessment Q3 (FOL Modeling)

Task: Write FOL for:

- Every student in this course has an ID.
- Riya is a student in this course.

Then ask: what can be concluded about Riya?

Assessment Q4 (Inference Strategy)

A doctor asks: *"Does patient P likely have condition C?"*

Prompt:

- Which is more natural first: forward chaining or backward chaining?
- Why?

Assessment Q5 (Representation Choice)

For a university knowledge system (courses, prerequisites, instructors, departments):

- What would you store as rules?
- What would you store as ontology classes/relations?
- Where would semantic network visualization help stakeholders?

Key Takeaways

1. Knowledge representation and inference are inseparable in AI design.
2. Propositional logic gives clean foundations; FOL adds object-level expressiveness.
3. Chaining and resolution provide complementary inference perspectives.
4. Rules, semantic nets, and ontologies serve different engineering needs.
5. Real systems often combine these representations, not just one.

Questions?